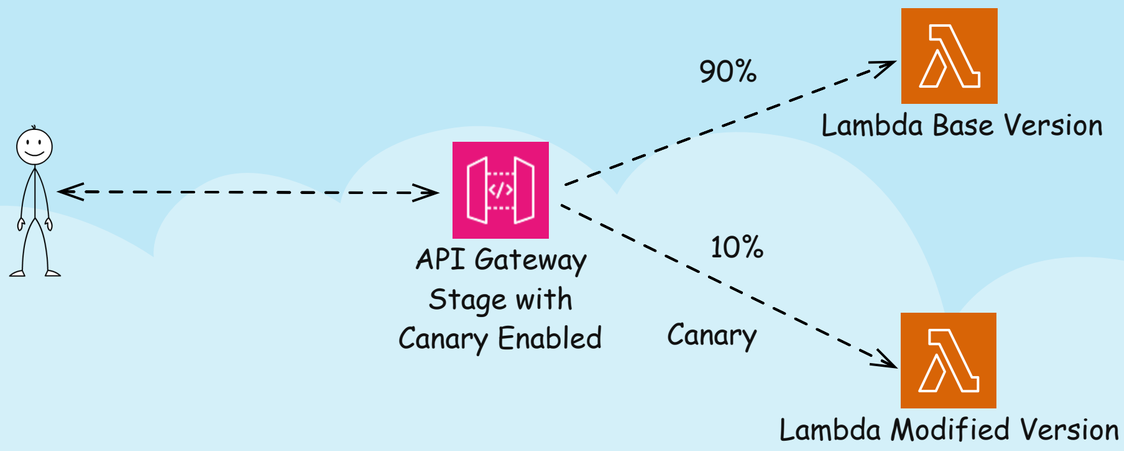
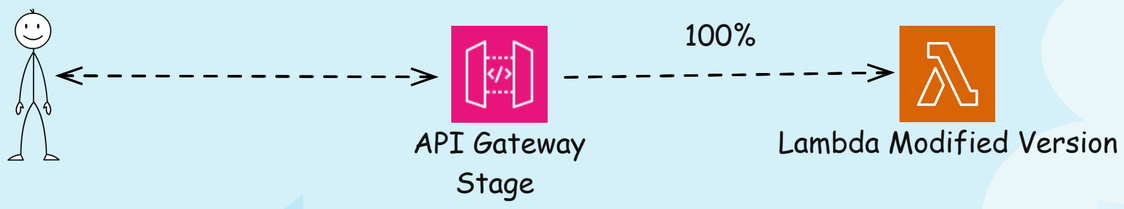
API Gateway Canary Deployment

CLOUD WITH RAJ
www.cloudwithraj.com



PROMOTE CANARY



Did you know - Amazon API Gateway provides canary deployment feature out of the box?

You can enable Canary within an API Gateway stage, and assign a % of the traffic to go to a different backend, such as modified Lambda

If things go bad with the % traffic testing, you can delete Canary and 100% of the traffic will fall back to original base Lambda

However, if testing with the % (10% in the diagram) traffic goes fine, then you can "promote" canary, then full traffic will be sent to the Lambda modified version and old Lambda integration will be deleted.

This is very handy because, you don't need to put canary logic in your code. This can also be controlled via AWS CLI commands or Infra as Code

AWS Gen AI Landscape

CLOUD WITH RAJ
www.cloudwithraj.com

Embedding and RAG

Bedrock (Knowledge Base)

OpenSearch Vector Database

Neptune for graphRAG Vector

Embedding Models (Amazon Titan , Cohere Embed)

Agentic AI

Bedrock (Using Action Groups)

Lambda (Invoke from Agents)

SageMaker AI

Strands Agent

Infrastructure (Train, and host your own models)

EC2 (Trainium, Inferentia, GPUs)

SageMaker AI

Notable LLMs (Hosted in Bedrock)

Amazon Nova & Titan

Anthropic Claude

A21 Labs

Cohere

Meta Llama

Stable Diffusion

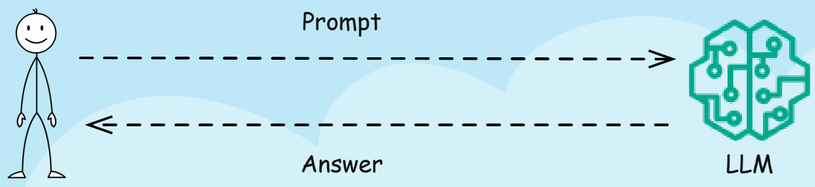
Coding & Productivity

Amazon Q (Coding and intelligence Assist)

Agentic AI Evolution

CLOUD WITH RAJ
www.cloudwithraj.com

Simple Prompting



Challenges

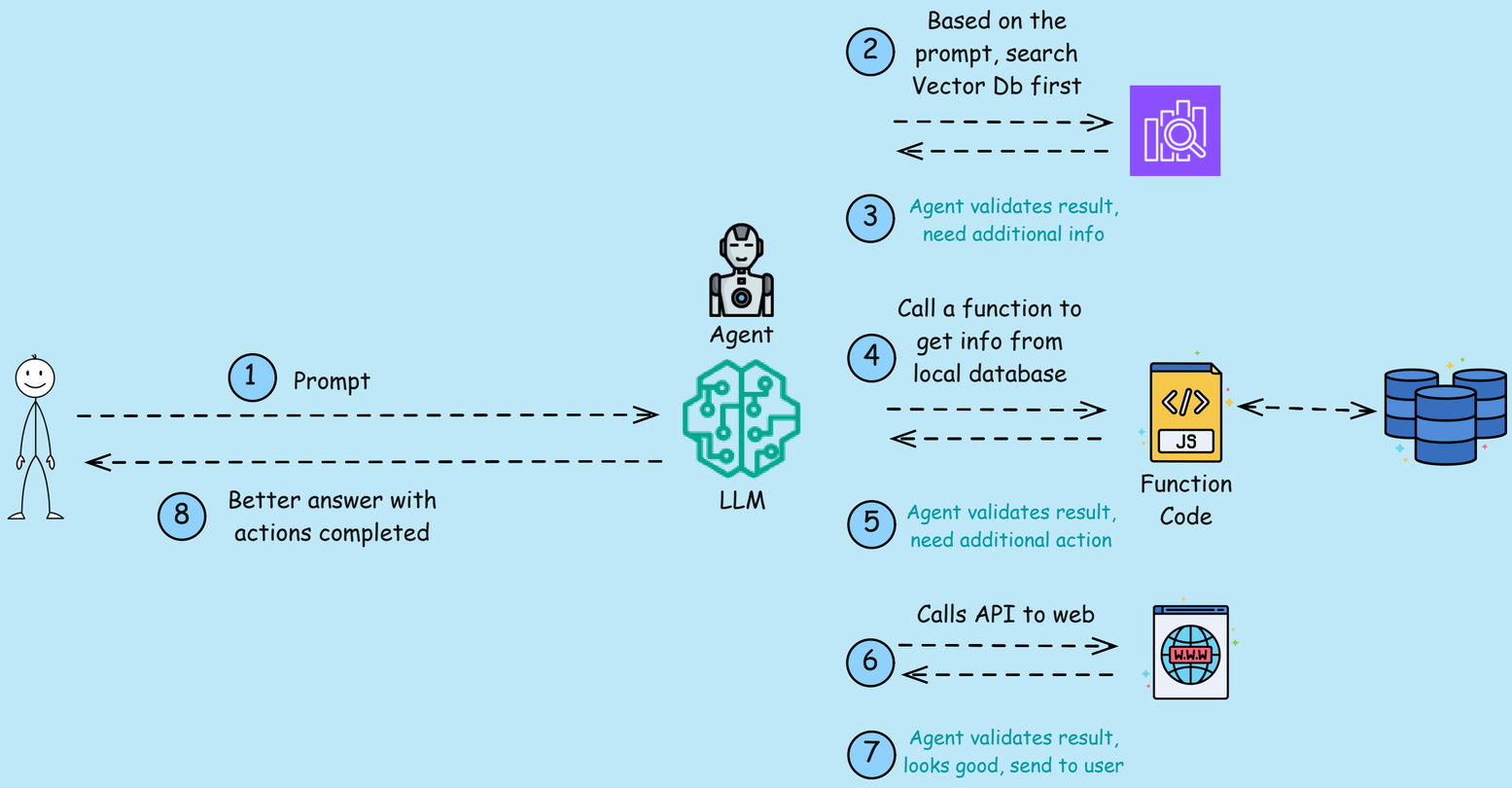
1. Static, pre-trained data
2. Not able to integrate project-specific data

RAG (Retrieval Augmented Generation)

Challenges

1. Once RAG gives you an answer, there is no way to validate, iterate, and refine the answer
2. Non dynamic access to data, heavily biased towards embedded vector database

Agentic AI



Advantages

1. Because you can invoke code, you can literally do anything from the LLM. You can even have multiple functions, and LLM can dynamically invoke them based on prompt

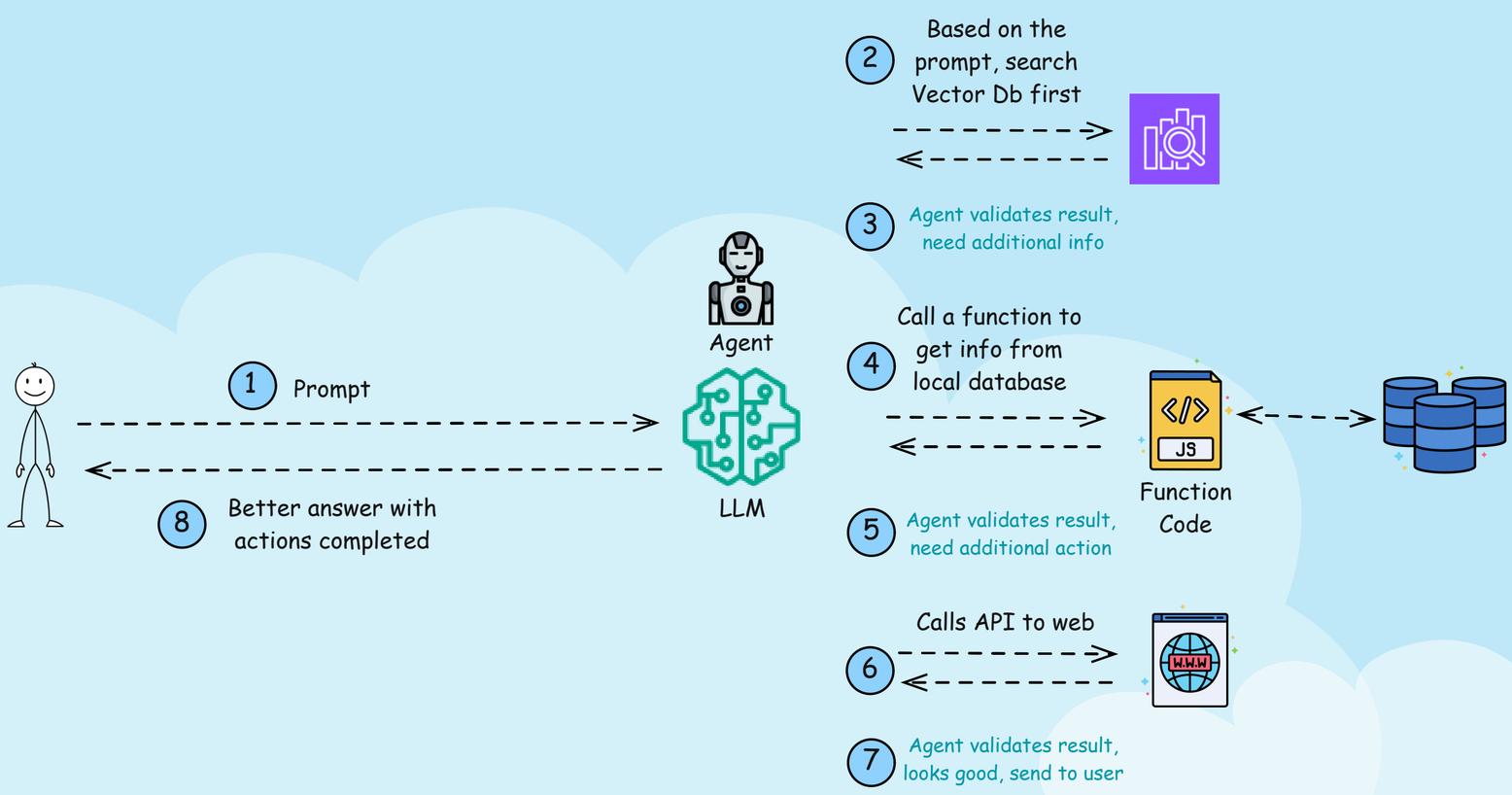
2. Iterative flow - LLM agent validates result and iterate and refine

3. Given the flexibility of calling API or code, you can perform actions e.g. book vacation based on budget, send emails based on meeting recroding etc.

4. This is the fundamental flow, you can even invoke multiple Agents and LLMs using Agentic AI

Agentic AI

CLOUD WITH RAJ
www.cloudwithraj.com



An AI agent is:

A program (or system) that takes input (observations or data)

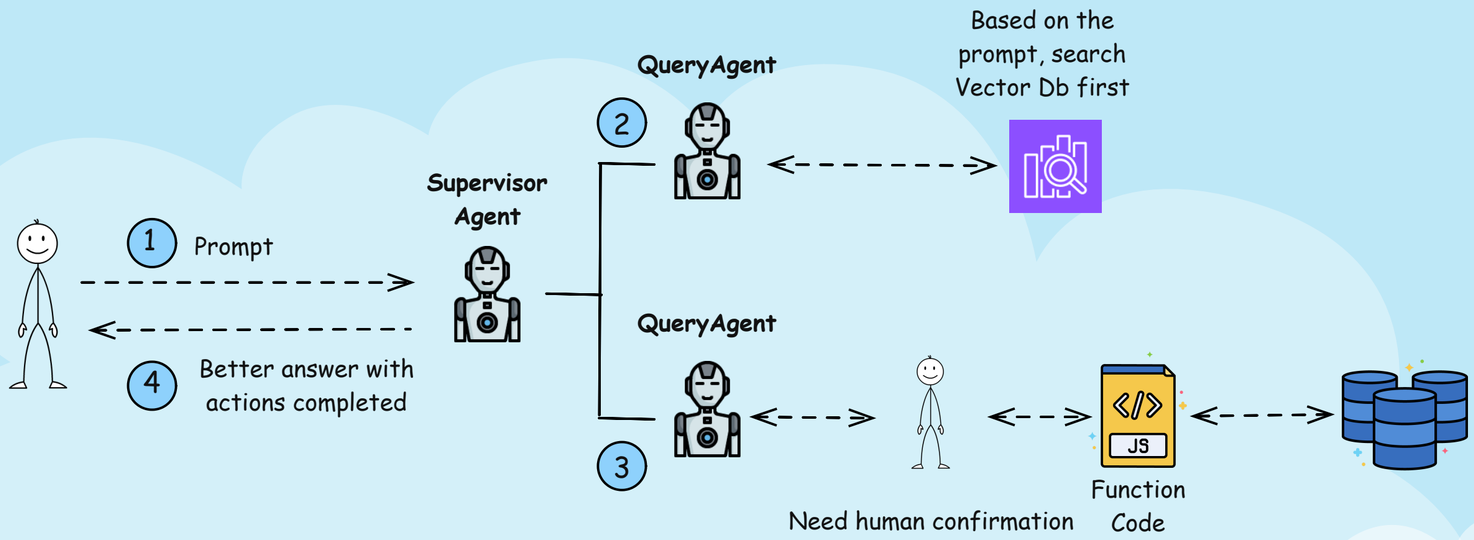
Makes decisions using AI models (like LLMs)

And performs actions (like calling tools, APIs)

Here is the biggest strength of AI agent - it can dynamically and autonomously call various tools, and go through iterations. For example, if the result of one tool call doesn't achieve the desired result, it will call another (or same) tool again. It is NOT just a static if-else loop, but a dynamic feedback loop, making it so powerful

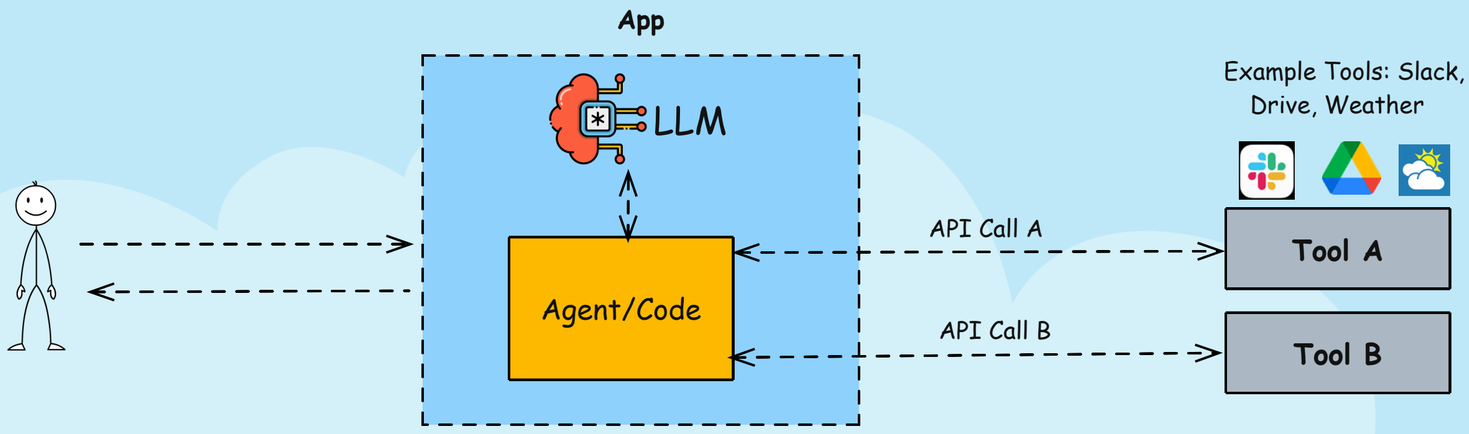
Multi Agent Architecture - Human In The Loop

CLOUD WITH RAJ
www.cloudwithraj.com



Challenges Before MCP (Model Context Protocol)

CLOUD WITH RAJ
www.cloudwithraj.com




```
WEATHER_API_URL =
"https://api.example.com/v1/weather"
headers = {"Authorization": "Bearer YOUR_API_KEY"}
payload = {"operation": "weather", "country": "usa", "zip":
12345}
```

How does the agent code interact with the weather tool? Via API. And to do that, as shown, the agentic code, needs to know the API URL, required header information, and payload.

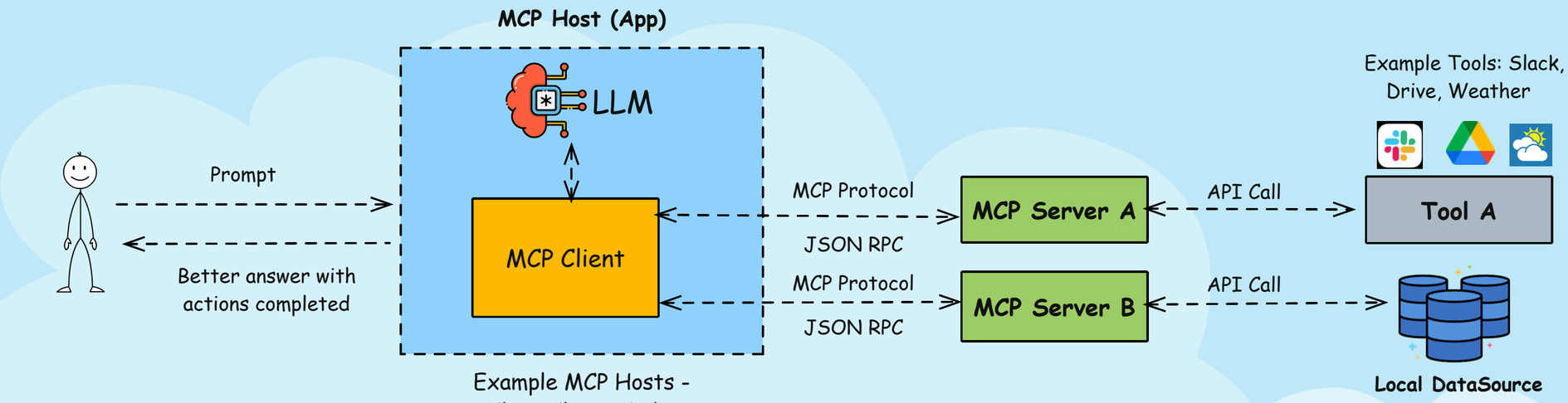
This works, but there are some challenges too:

1. If developers in the weather app changes the API schema, and adds/removes input/output fields, and agentic code doesn't know about it, the connection breaks
2. Imagine, in real-world, agents talk to numerous such tools. Each tool requires separate integration with custom code
3. It's a tedious task for the app developers to instrument all these separate tools inside the app code

Hence MCP (Model Context Protocol) was born to standardize the communication between Agents and tools. Let's find out about MCP next!

MCP (Model Context Protocol)

CLOUD WITH RAJ
www.cloudwithraj.com

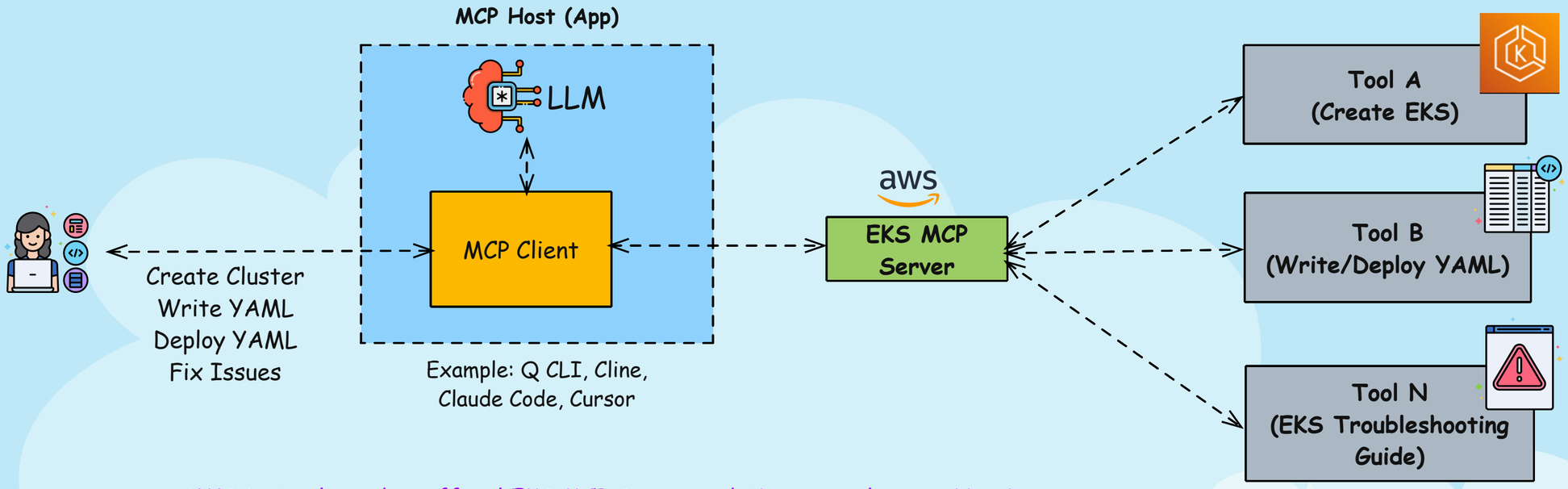


MCP standardizes the communication between the agentic code and tools (and local datasources, but tool is the most widely used). What does this mean?

1. An MCP client (think of a piece of code running inside the agent), connects to the MCP server, instead of connecting to the tool directly with a predefined API URL
2. The developers of each tool expose this MCP server
3. MCP client asks the server, "What can you do?". In response, the MCP server responds with the tool capability, description, and input/output schemas
4. IMPORTANT - this discovery is dynamic, and happening at runtime. If input/output field changes, this discovery call will reveal all the fields at runtime
5. The MCP client registers all these, and then can invoke the tool via the MCP server
6. The MCP server handles the connection to the tool. As a result, the code does NOT need to hardcode the API URLs like before
7. The developers of the tools give you the code for the MCP server. You still need to pass authn/z parameters (for e.g. token and team ID for Slack Tool, Access Key and Secret Access Key for AWS Tool etc) to the MCP server. BUT you can run this MCP server locally, or on the cloud. It's up to you

Official AWS EKS MCP Server

CLOUD WITH RAJ
www.cloudwithraj.com



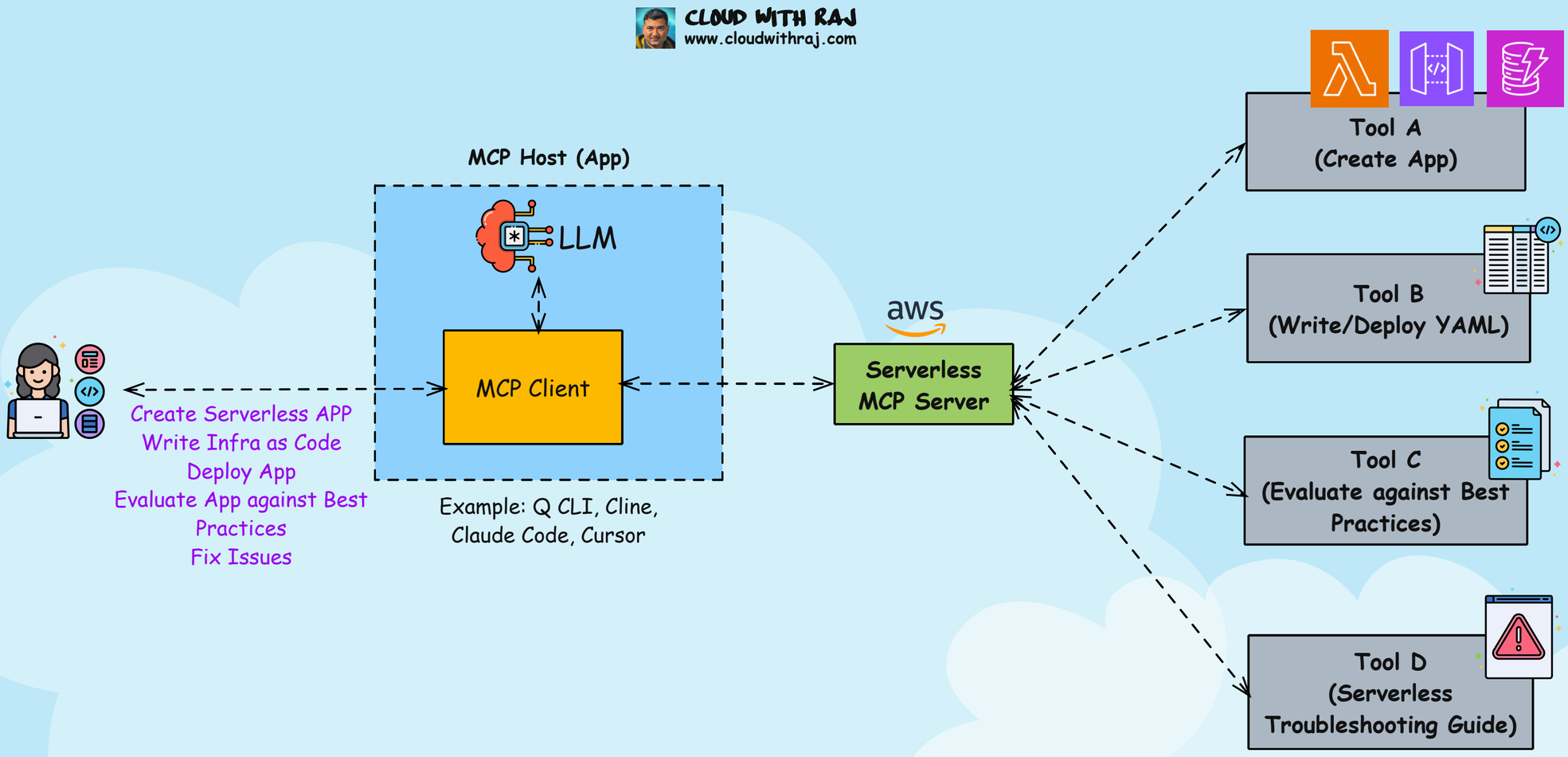
AWS just released an official EKS MCP Server and it's a gamechanger. Here's what it does:

- Creates EKS clusters autonomously
- Writes and edits Kubernetes manifests in your IDE
- Trained on years of real AWS EKS troubleshooting info. This MCP Server Fixes issues super fast
- All controlled with plain English

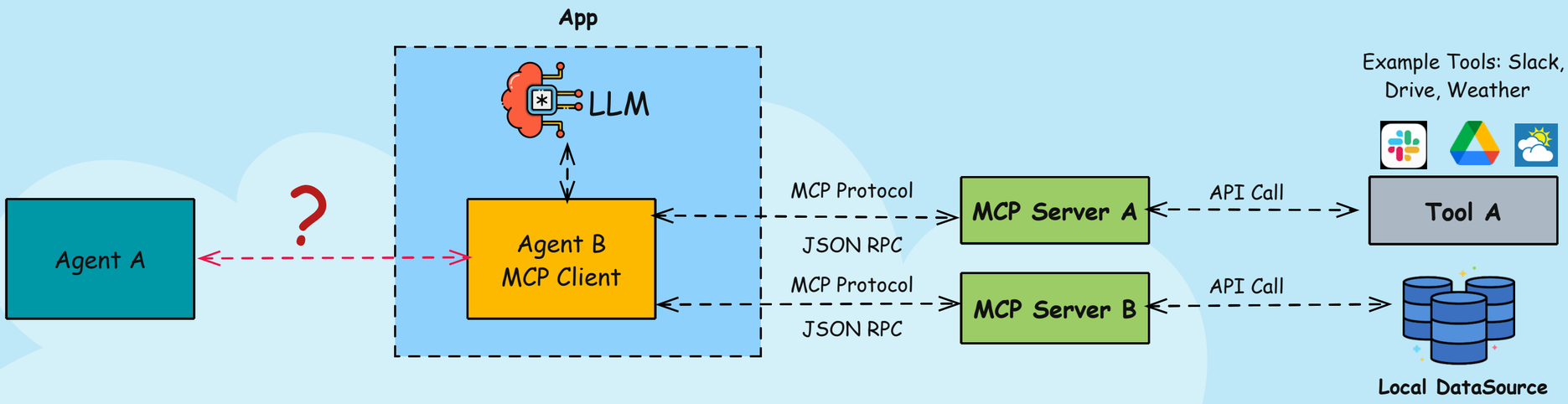
If you're a DevOps, SRE, or Cloud Engineer, learn this before your next interview.

Official AWS Serverless MCP Server

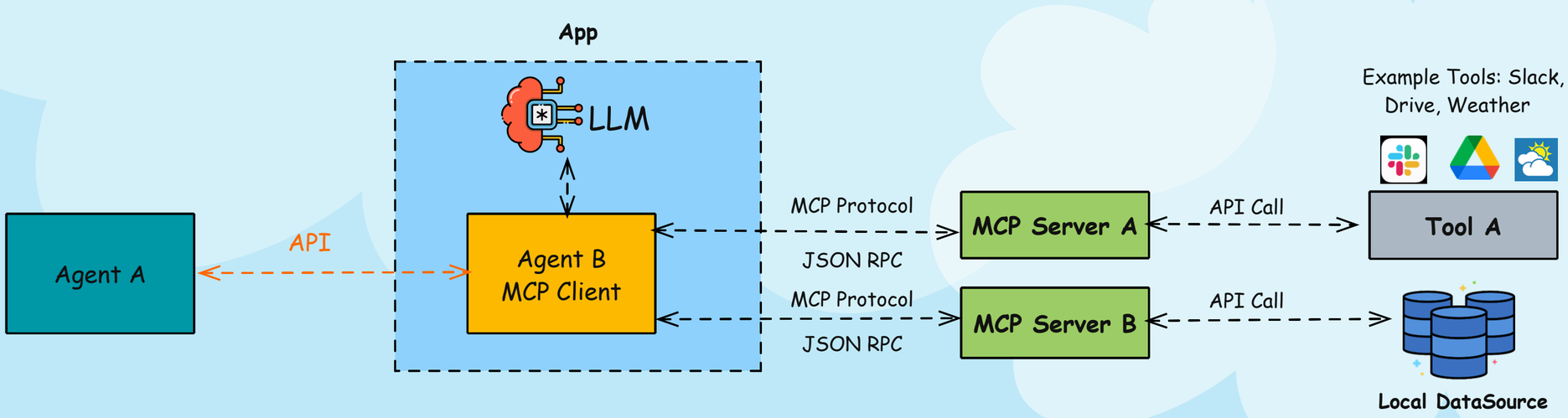
CLOUD WITH RAJ
www.cloudwithraj.com



Challenges Before A2A (Agent 2 Agent)



MCP handles the communication between the agent and tools (and local datasources). But how about agent-to-agent?



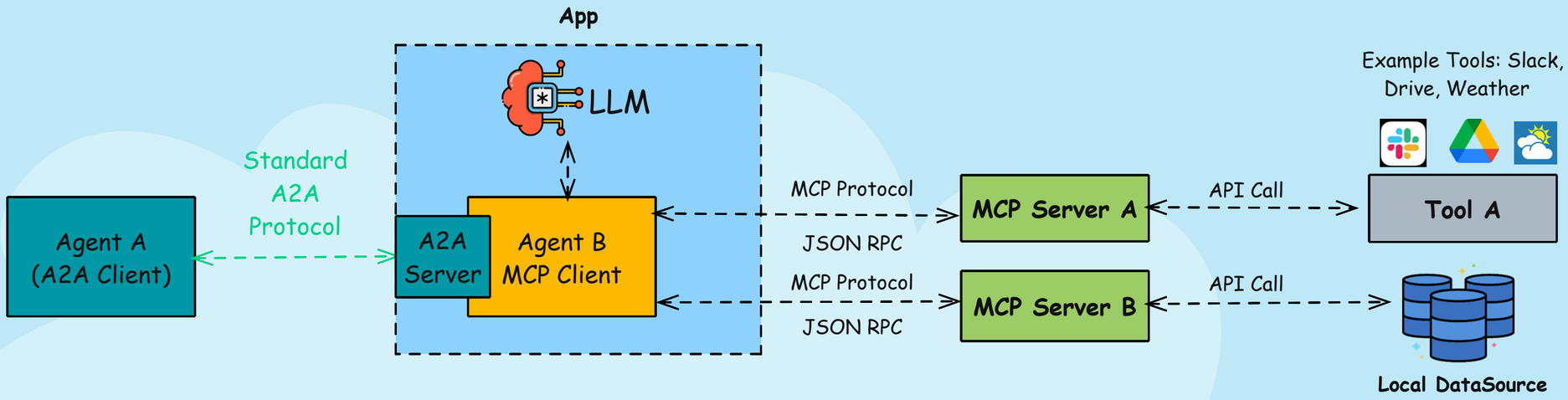
Similar to any other API call, Agent A will invoke the API URL of Agent B, and pass AuthN/Z parameters, and a payload. This brings similar challenges:

- Each agent needs to know what the other agent does beforehand
- Hardcode the connection URL and authn/z parameters
- Have to know how to submit prompts/tasks to the other agent

A2A (or Agent2Agent) standardizes the communication between agents. What does this mean? Let's find out next!

A2A (Agent 2 Agent)

CLOUD WITH RAJ
www.cloudwithraj.com



1. Agent A, runs a A2A client, which connects to Agent B A2A Server

2. A2A server returns an Agent Card to Agent A

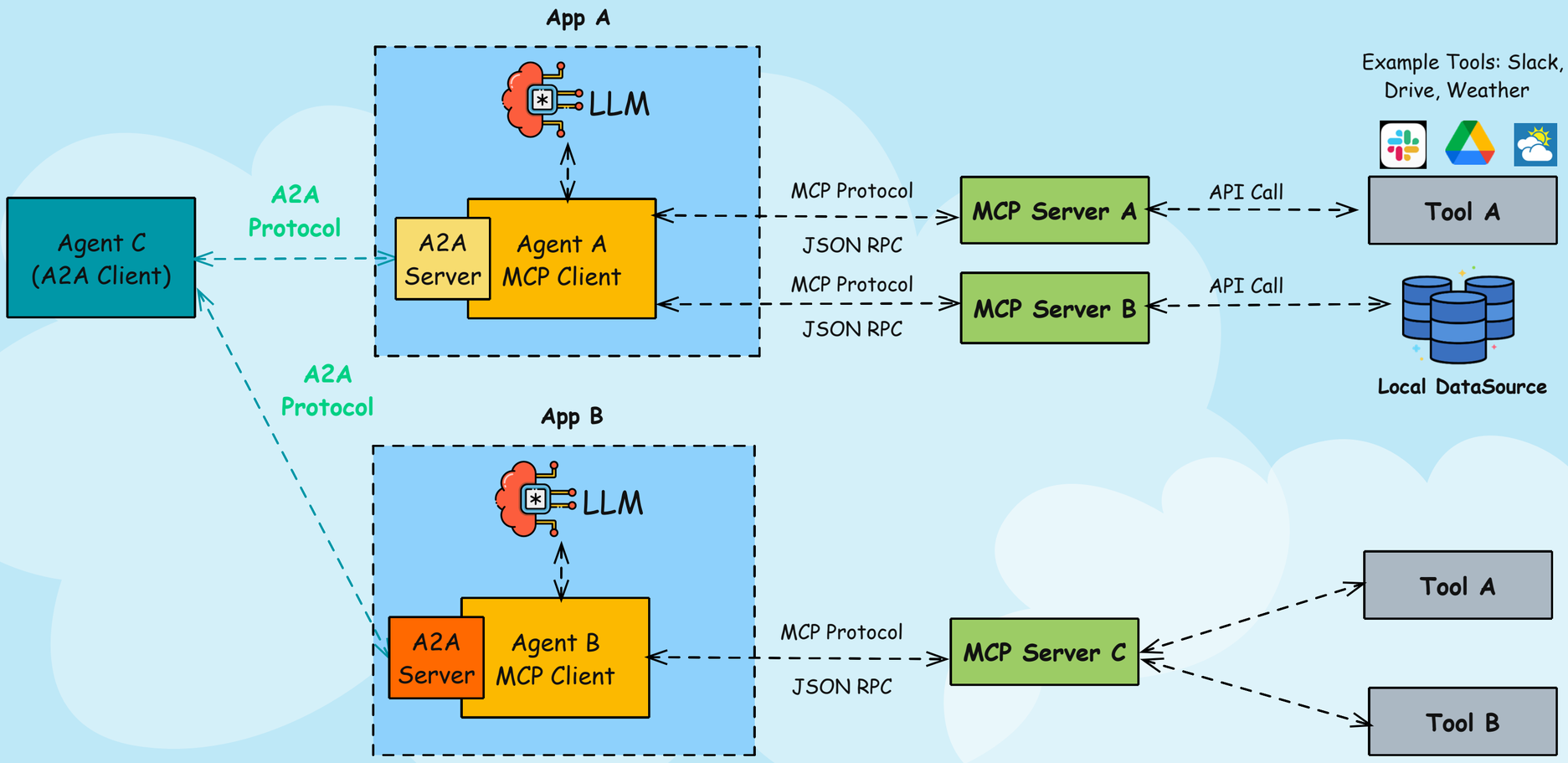
3. This Agent card includes capabilities of Agent B, endpoint URL, how to submit tasks/prompts, how to get notifications on task status, and authn/z mechanism

4. IMPORTANT - Note how dynamic this is (similar to MCP). Using this any agent can discover capabilities, and connection URL, and other info during runtime, and don't need to hardcode beforehand

5. A2A do NOT include payload schema at this time (MCP returns payload schema for the tools/datasources)

A2A (Agent 2 Agent) + MCP (Model Context Protocol)

CLOUD WITH RAJ
www.cloudwithraj.com



A2A + MCP Flow

1. Agent A, runs a A2A client, which connects to Agent B A2A Server

2. A2A server returns an Agent Card to Agent A

3. This Agent card includes capabilities of Agent B, endpoint URL, how to submit tasks/prompts, how to get notifications on task status, and authn/z mechanism

4. IMPORTANT - Note how dynamic this is (similar to MCP). Using this any agent can discover capabilities, and connection URL, and other info during runtime, and don't need to hardcode beforehand

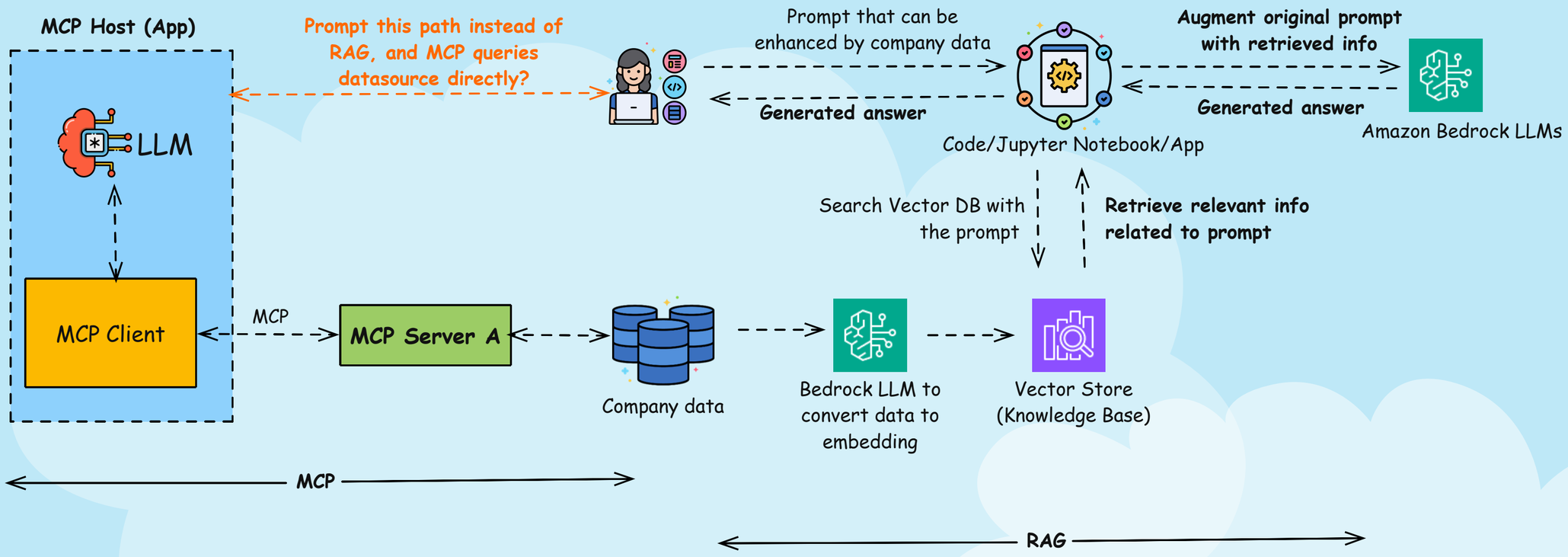
5. A2A do NOT include payload schema at this time (MCP returns payload schema for the tools/datasources)

In summary, MCP standardizes the connection between LLM Agents and tools, and A2A standardizes the connection between two agents. They work hand in hand because they complement each other and do not compete with each other.

Detailed Video:

Can MCP Replace RAG?

CLOUD WITH RAJ
www.cloudwithraj.com



The obvious question on your mind is, well, if MCP can access the datasource, and the same datasource is being used in RAG, why don't I skip the whole RAG part, and just ask my question to the MCP Host App as shown above? There are a couple of problems with this approach:

- If you want MCP to connect to the database directly, that means the app team needs to give access to the database to the MCP server.
- If you have ever worked at an enterprise, you know teams don't want to give access to their actual database. You might think this is territorial (part of it is!), but there are also good reasons for it:
- If one team is granted access, more teams will be granted access
- As these teams access the actual database, it eats away at the read and write capacity for the business application to serve customers
- Database is expensive, and scaling up means more cost
- App team, managing the database, can't change the schema freely because it'd break the MCP flow

RAG actually solves this problem, how?

- RAG flow does NOT access the actual database, but the vector database. This way, RAG queries don't consume database capacity
- The embedding process typically runs nightly during off-hours
- Embedding into a vector database also indexes the data, ideal for text queries
- The vector database is not impacted even if the schema is changed in the underlying database

As we can see, RAG also has some strengths. And MCP is already powerful. What now? Turns out there is a middle ground, a best-of-both-worlds solution.

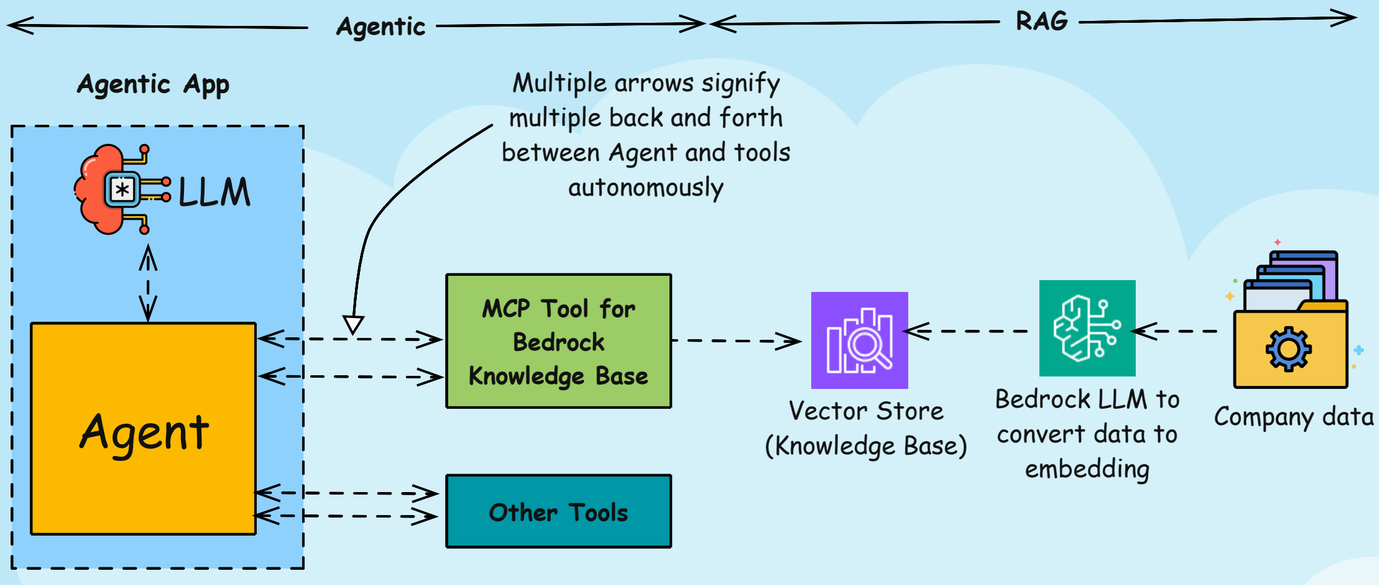
Sometimes you do want to query the MCP host because the MCP host has access to many tools, and even other agents! In those cases, if you need to utilize the vector database, MCP has a server that interacts directly with the vector database! For AWS, such MCP server can interact directly with Bedrock Knowledge Base which is basically the vector database.

On the other hand, if you need the RAG flow, you can go the RAG route. RAG route is generally faster because it just queries the vector database. Meanwhile, for the MCP flow, handshaking is involved, as discussed above in the brief MCP overview. This back and forth introduces some latency.

In summary, RAG is going nowhere, and MCP complements RAG! If you get this question in your interview or projects, knock it out of the park!

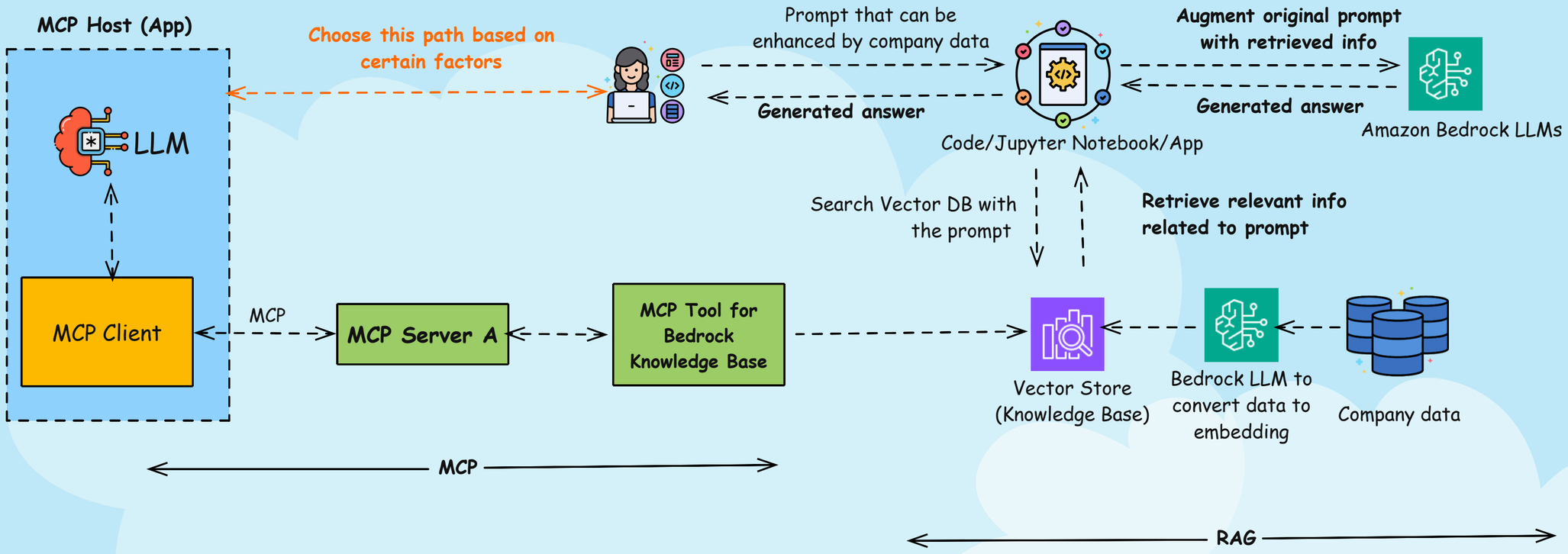
Agentic RAG

CLOUD WITH RAJ
www.cloudwithraj.com



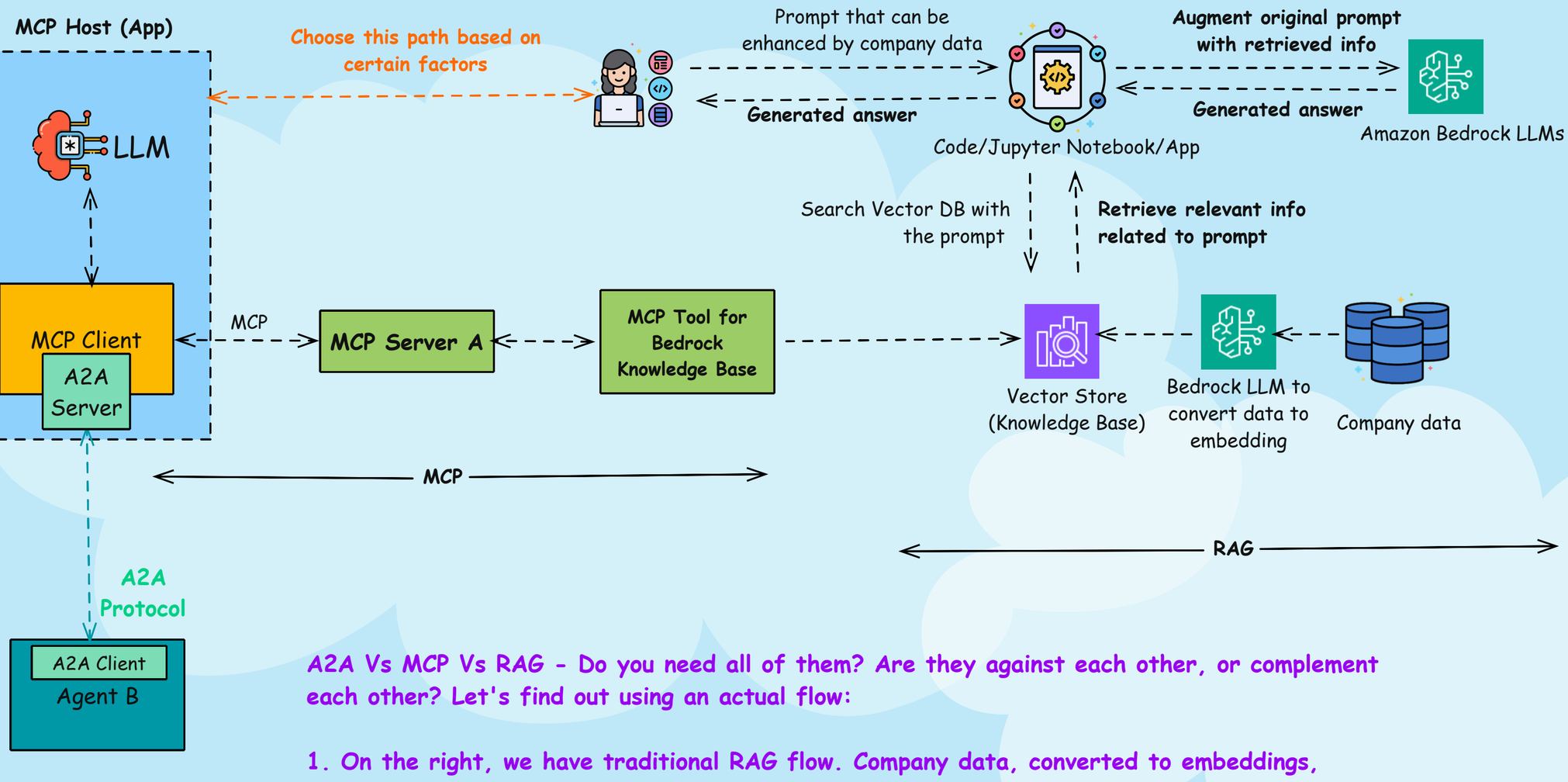
MCP with RAG

CLOUD WITH RAJ
www.cloudwithraj.com



MCP with RAG with A2A

CLOUD WITH RAJ
www.cloudwithraj.com



A2A Vs MCP Vs RAG - Do you need all of them? Are they against each other, or complement each other? Let's find out using an actual flow:

1. On the right, we have traditional RAG flow. Company data, converted to embeddings, stored in a vector store (Knowledge Base in Bedrock), and used in RAG

2. MCP actually makes RAG easier! Using MCP for Knowledge Bases, you can connect to this Knowledge Base directly

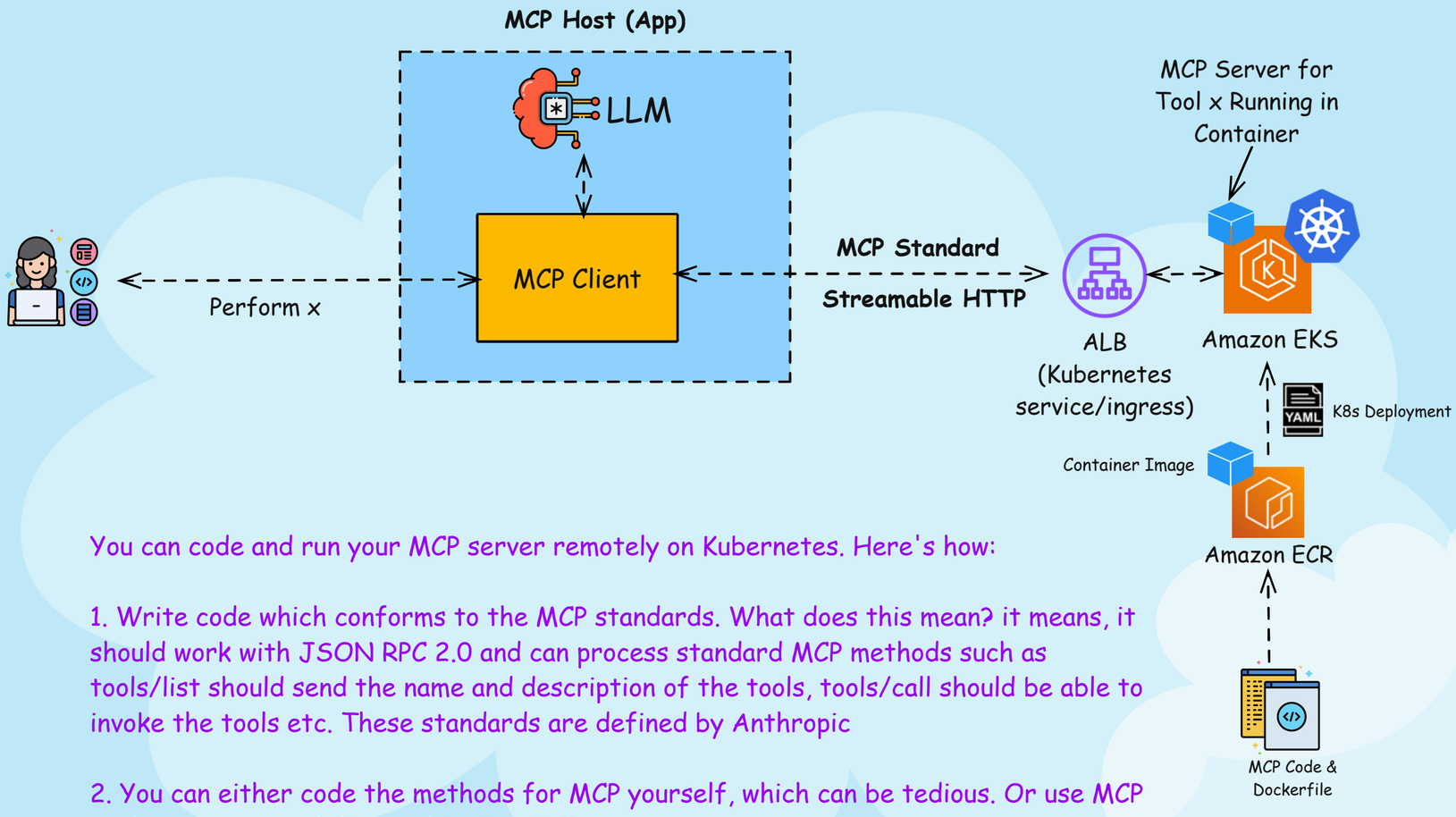
3. You can also bypass RAG and just invoke MCP Host when needed

4. A2A or Agent to Agent has nothing to do with connecting tools (such as Knowledge Base shown here). It facilitates one agent communicating with another.

5. IMPORTANT - Note how dynamic MCP, and A2A makes it. They can dynamically discover tools and agent capabilities, respectively, and act on it.

Remote MCP Server

CLOUD WITH RAJ
www.cloudwithraj.com



You can code and run your MCP server remotely on Kubernetes. Here's how:

1. Write code which conforms to the MCP standards. What does this mean? it means, it should work with JSON RPC 2.0 and can process standard MCP methods such as tools/list should send the name and description of the tools, tools/call should be able to invoke the tools etc. These standards are defined by Anthropic

2. You can either code the methods for MCP yourself, which can be tedious. Or use MCP implementation with library like FastMCP

3. Run Dockerfile to create a container and save it in ECR

4. Deploy to the Kubernetes cluster such as in Amazon EKS, and expose it using a ALB via service or ingress. ALB and container support streamable HTTP out of the box, and works nicely

5. Invoke your MCP Server using the ALB Url. For Streamable HTTP, you need to initialize a session, and then use the sessionID for subsequent calls to do tool discovery, and tool calls

Demo with actual code snippet:

AWS Strands Agent

CLOUD WITH RAJ
www.cloudwithraj.com

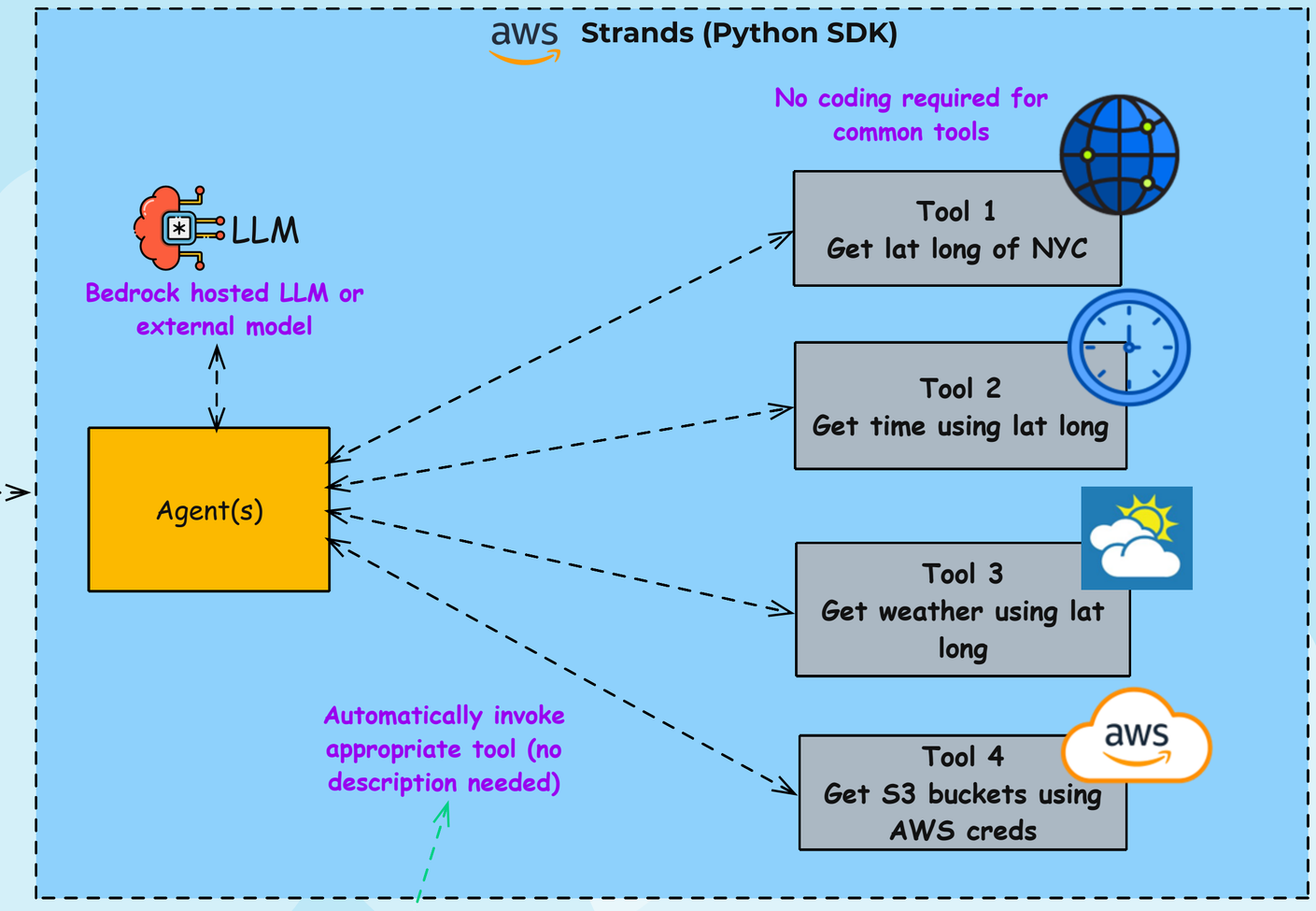
Think of AWS Strands as a single Python program that can implement powerful agentic workflows with a minimum amount of code. Why?

- Supports both Bedrock and external LLMs (e.g., Anthropic Claude)
- This one is awesome - it comes with 20+ prebuilt tools, that you do NOT have to code. For example:current_time (gives current time), use_aws (uses boto3 under the hood to list AWS resources), http_request (figures out API calls automatically)
- You can create your own custom tool
- It has MCP client built in, allowing you to connect with MCP Servers

Demo with actual code snippet:

AWS Strands Agent Code Snippet

CLOUD WITH RAJ
www.cloudwithraj.com




```python
from strands import Agent
from strands.models.anthropic import AnthropicModel
from strands_tools import current_time, http_request, use_aws

# Uses Bedrock Claude if no model given, can also use external model if specified

WEATHER_SYSTEM_PROMPT = """You are a weather assistant with HTTP capabilities. You can:
1. Make HTTP requests to the National Weather Service API
2. Process and display weather forecast data
3. Provide weather information for locations in the United States

When displaying responses:
- Format weather data in a human-readable way
- Highlight important information like temperature, precipitation, and alerts
- Handle errors appropriately
- Convert technical terms to user-friendly language

Always explain the weather conditions clearly and provide context for the forecast.

# Create agent with Claude 4
agent = Agent(
    tools=[current_time, http_request,use_aws],
    system_prompt=WEATHER_SYSTEM_PROMPT
)

response = agent("1. what is the time in new york city? 2. What is the weather in new york city? 3. list the s3 buckets in my aws account")
print(response)

```

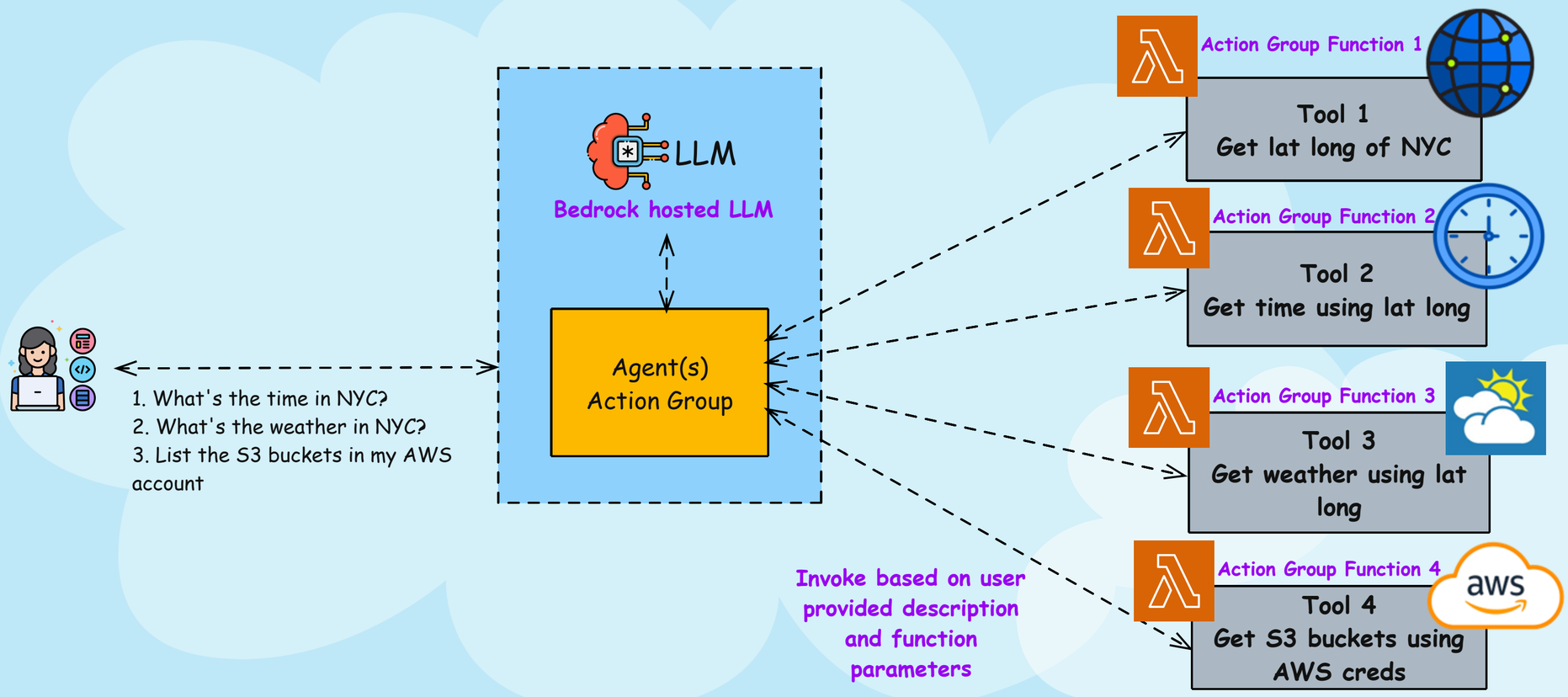
Note: I am just invoking pre-built tools and not coding

Demo with actual code snippet:

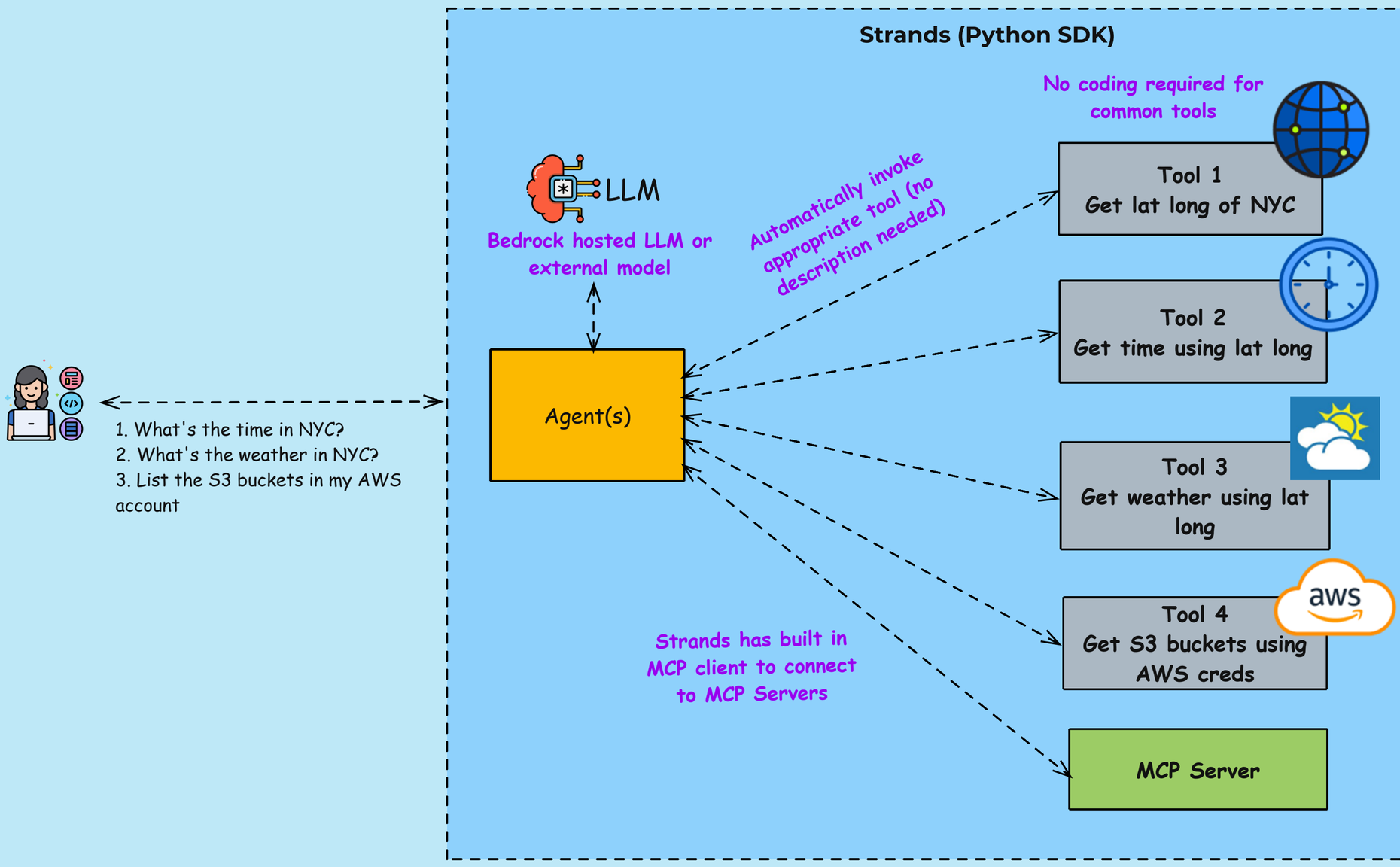
Strands Vs Bedrock Agents

CLOUD WITH RAJ
www.cloudwithraj.com

Bedrock Agents

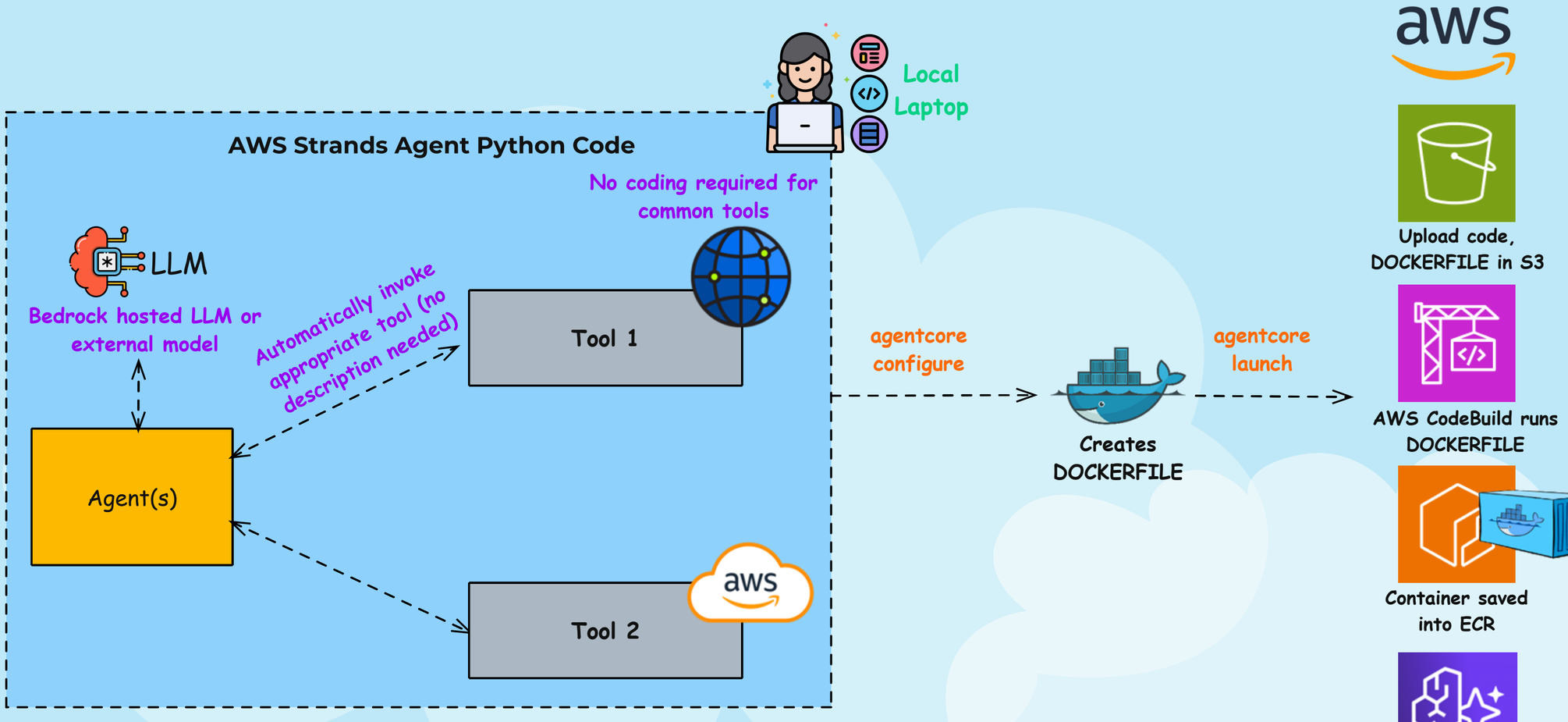


aws Strands Agents



Strands Vs Bedrock Agentcore

CLOUD WITH RAJ
www.cloudwithraj.com



What is the difference between Strands Agents, and Bedrock Agentcore?

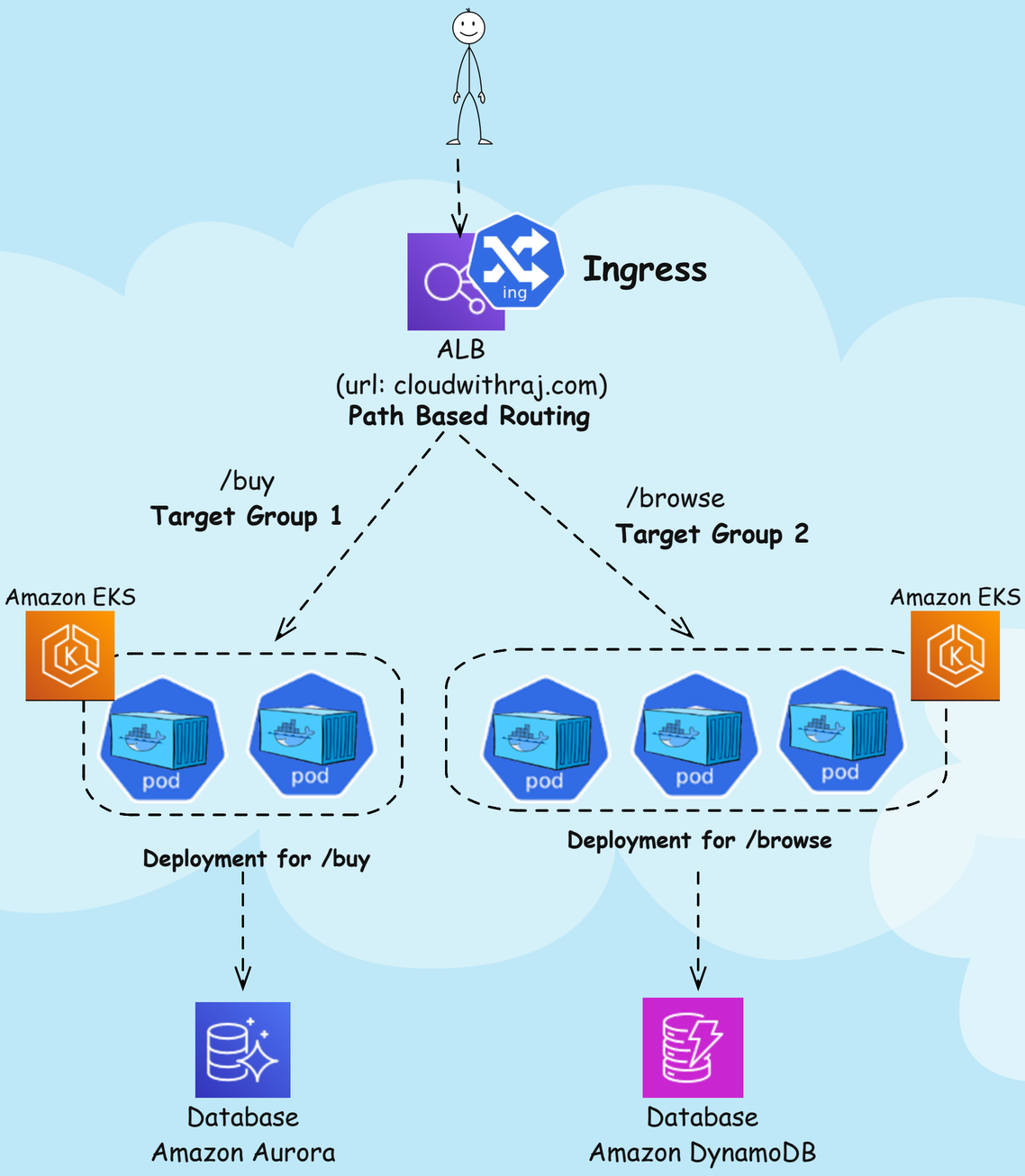
Think of AWS Strands as a single Python program that can implement powerful agentic workflows with a minimum amount of code.
- Supports both Bedrock and external LLMs (e.g., Anthropic Claude)
- This one is awesome - it comes with 20+ prebuilt tools, that you do NOT have to code. For example:current_time (gives current time), use_aws (uses boto3 under the hood to list AWS resources), http_request (figures out API calls automatically)
- You can create your own custom tool
- It has MCP client built in, allowing you to connect with MCP Servers

At this point, you have your agent running in your laptop. Amazon Bedrock Agentcore helps you run this agent on AWS!
- Run command "agentcore configure" and it will create a DOCKERFILE to containerize your agent code
- Run command "agentcore launch" and it will upload and run your agent from local to Agentcore in AWS!
- The underlying infra is Serverless
- Logs, metrics, traces are automatically configured and provided
- AuthN/Z can be added easily

Demo with actual code snippet:

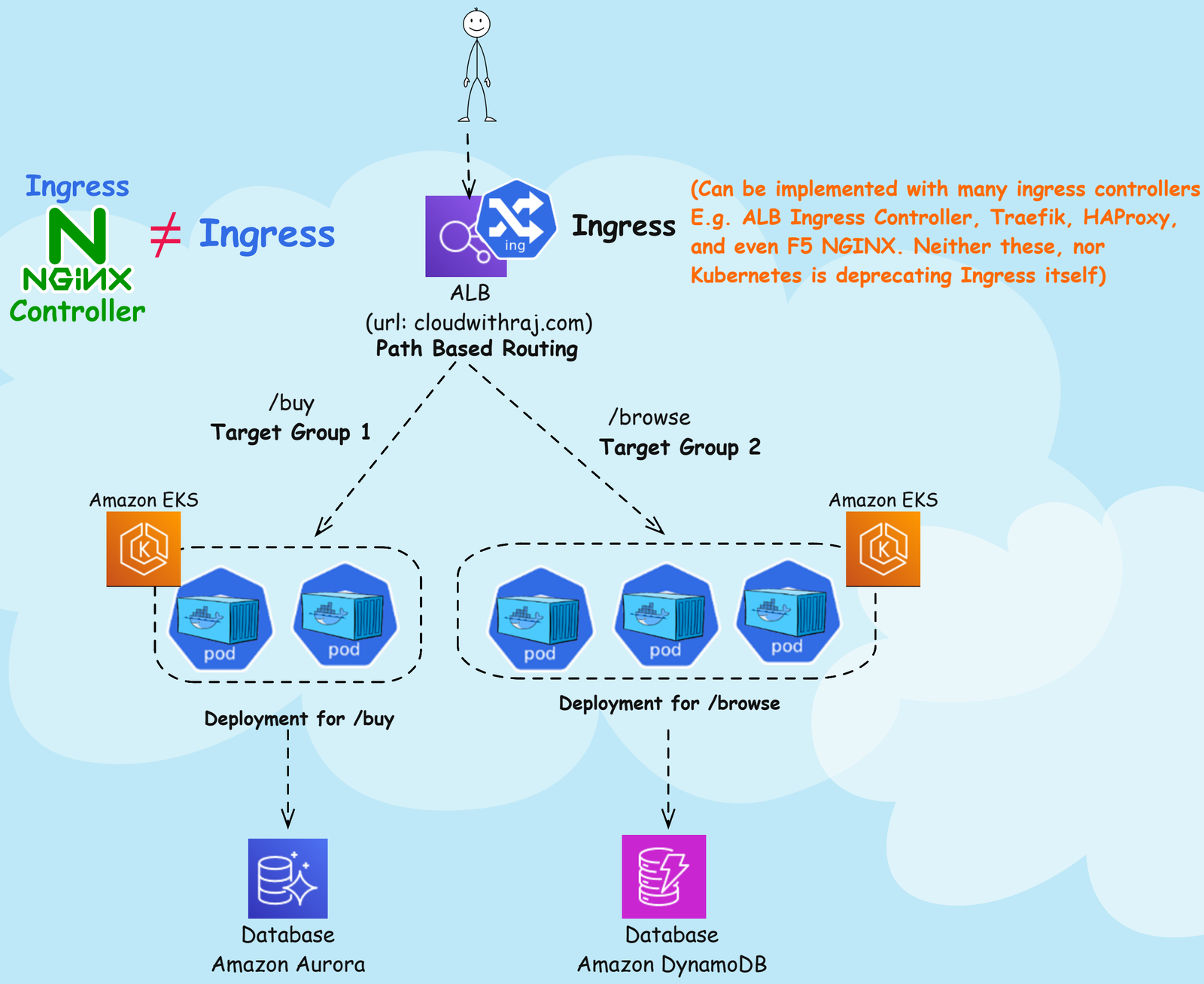
Microservice on Kubernetes

CLOUD WITH RAJ
www.cloudwithraj.com



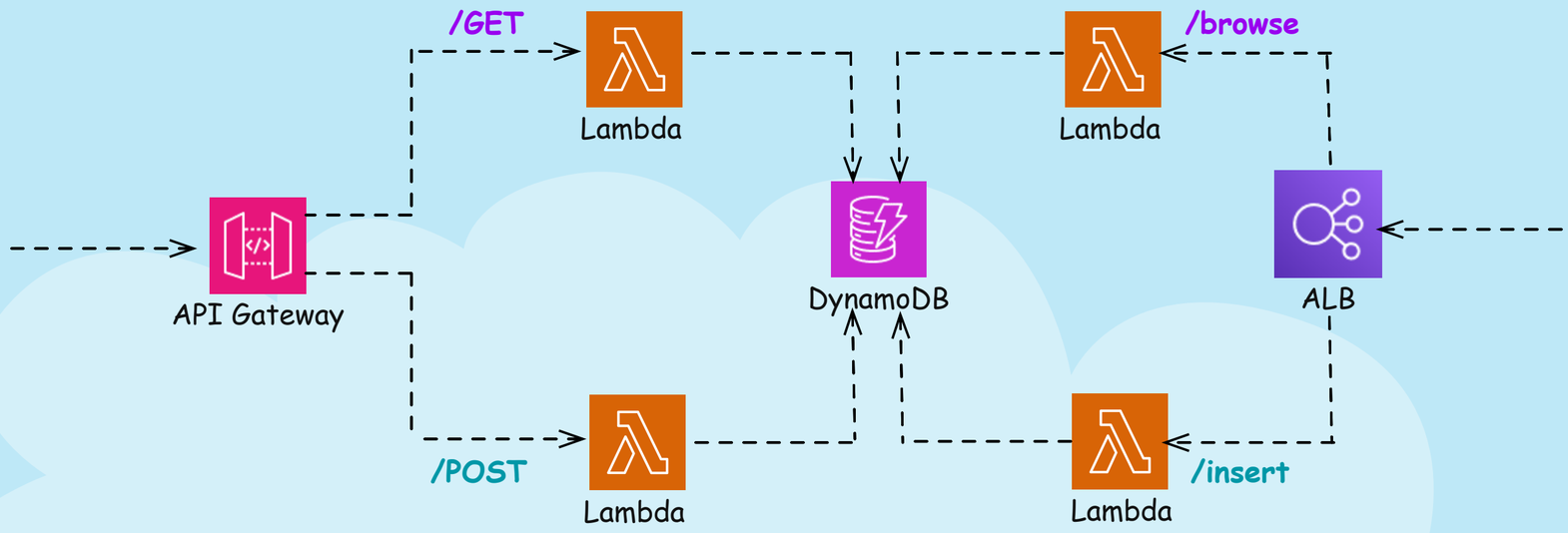
Kubernetes Ingress NOT Deprecated

CLOUD WITH RAJ
www.cloudwithraj.com



ALB VS API Gateway

CLOUD WITH RAJ
www.cloudwithraj.com



In AWS, two services often confuse architects: Application Load Balancer (ALB) and Amazon API Gateway. On the surface, both send traffic at OSI Layer 7 and scale automatically. But their use cases are very different.
Let's break down the differences you need to know for real-world design and interviews.

1. Routing
API Gateway routes by HTTP method (e.g., GET → one Lambda, POST → another).
ALB routes by path (e.g., /browse vs /insert), independent of method, out of the box

2. Security
API Gateway enforces HTTPS by default with AWS-managed certs.
ALB defaults to HTTP - you must integrate with ACM to enable HTTPS.

3. Advanced Controls
API Gateway: Usage plans, quotas, rate limiting, request validation, response mapping, and DDoS protection built in.
ALB: Needs WAF/Shield for rate limiting and DDoS protection, backend code for validation.

4. Deployment
API Gateway: Canary deployments, multi-account/multi-region Lambda integration, OpenAPI spec import/export.
ALB: No canary support, region/account-bound targets, no simple spec migration.

5. Service Integration
API Gateway: Though integrates with many AWS services directly, doesn't integrate with ASG and Kubernetes directly
ALB: Integrates with EC2, ASG, Lambda. Native integration with Kubernetes, can act as an ingress or a Kubernetes service

6. Timeouts & Health Checks
API Gateway: 30s default timeout (can be increased with support request). No built-in health checks.
ALB: 4,000s timeout, health checks out-of-the-box.

7. Cost Model
API Gateway: Serverless, pay-per-use.
ALB: Incur charges even when idle, though relatively small at average traffic.

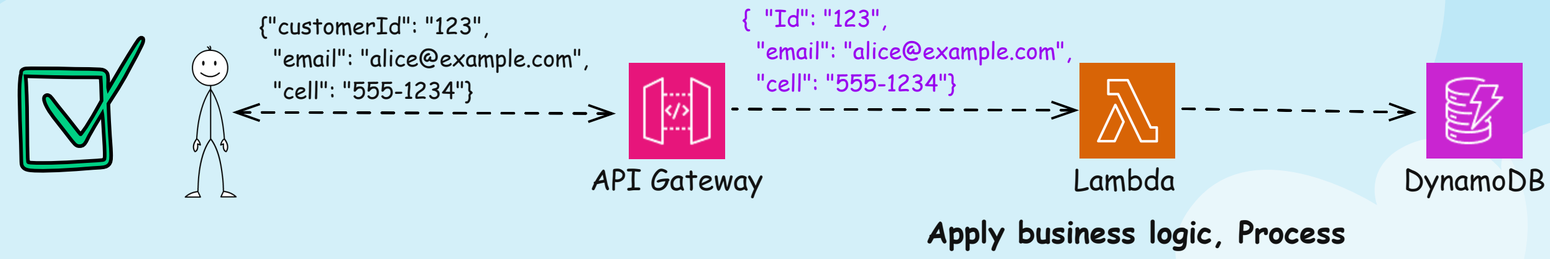
API Gateway Payload Transformation

CLOUD WITH RAJ
www.cloudwithraj.com

Lambda expects fieldname "Id" instead of "customerId". Rejects because field name mismatch

API Gateway can transform fields using Velocity Template Language (VTL)

{"Id": "$input.path('$.customerId')"}



Apply business logic, Process transaction

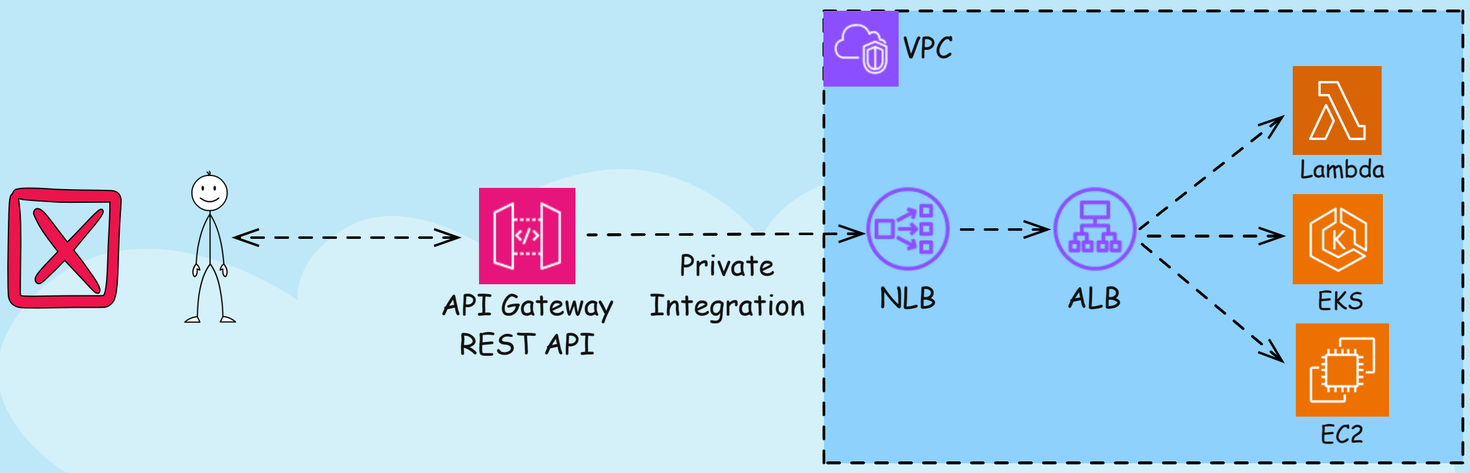
Did you know that API Gateway can also do payload transformation?

If the application is invoking the Lambda, sending the payload with certain field names, but your backend expects different names, you can achieve that in the Amazon API Gateway using Apache Velocity Template language.

In the above example, the producer is sending a fieldname "customerId", and the Lambda is expecting "Id". API Gateway is transforming the payload accordingly, and the Lambda is processing the transaction, instead of rejecting it.

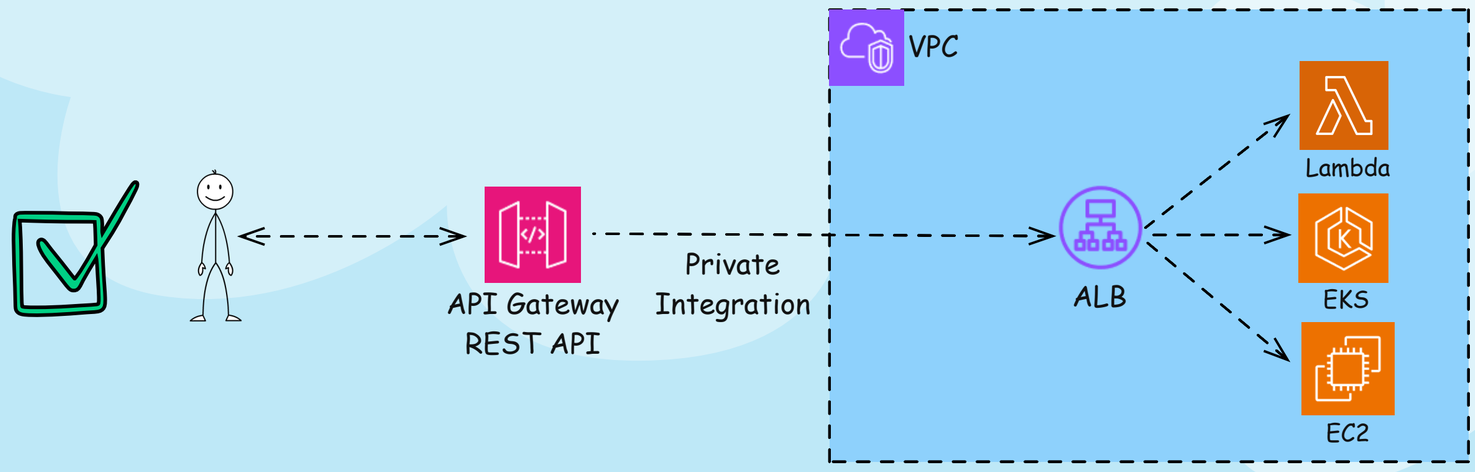
API Gateway REST API Private Integration

CLOUD WITH RAJ
www.cloudwithraj.com



REST API Private Integration could only terminate at NLB in VPC. If app is exposed through ALB, required NLB to ALB hop/complexity. HTTP API can terminate in ALB directly

Re:Invent 2025 Changed This!

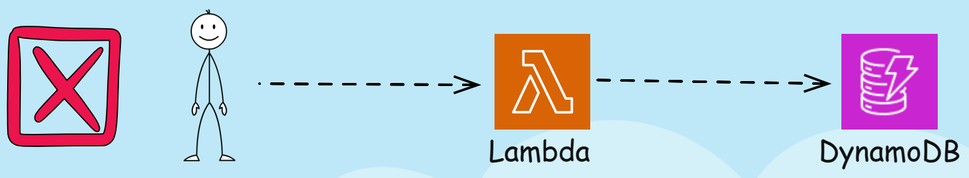


Now, REST (and HTTP API) don't need a NLB inbetween, they can go to ALB directly for Private API - cheaper, simpler, and faster

DynamoDB Global Secondary Index Change

CLOUD WITH RAJ
www.cloudwithraj.com

BEFORE



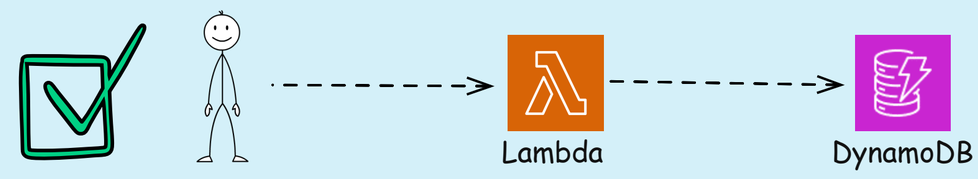

```
{
"Artist":"Death Cab for Cutie",
"SongTitle":"I will follow you into the dark",
"AlbumTitle":"Plans",
"SongRating":4.9,
"Genre":"Indie rock",
"Year":2005
}
```

Primary Key
Partition Key
Sort Key
Global Secondary Index
Partition Key
Sort Key

GSI can only have two fields

Can only query based on Primary Key (Artist, SongTitle) or GSI (Genre, Year)

AFTER Re:Invent 2025




```
{
"Artist":"Death Cab for Cutie",
"SongTitle":"I will follow you into the dark",
"AlbumTitle":"Plans",
"SongRating":4.9,
"Genre":"Indie rock",
"Year":2005
}
```

Primary Key
Partition Key
Sort Key
Global Secondary Index
Sort Key #1
Partition Key
Sort Key #2

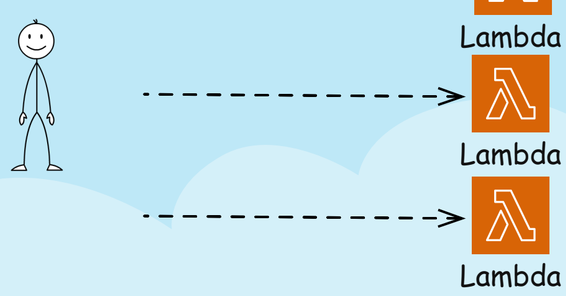
GSI can only have upto FOUR fields (up from two) - 2 in Partition Key, 2 in Sort Key (Up from 1 each). No change in Primary Key

More flexibility in query

Lambda Managed Instances

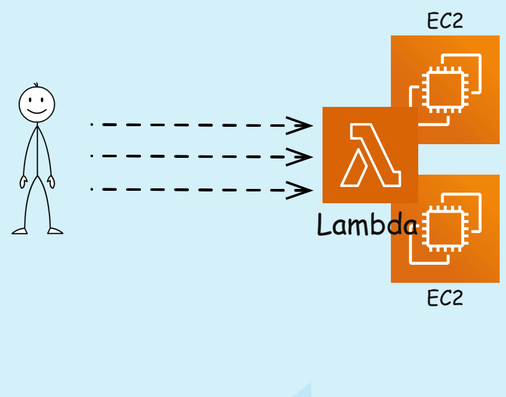
CLOUD WITH RAJ
www.cloudwithraj.com

BEFORE



- No option to select underlying hardware
- One Lambda instance runs one transaction at a time
- Concurrent transactions spin up multiple Lambda instances
- Cold Start on each instance by default
- High Availability without extra cost

AFTER Re:Invent 2025

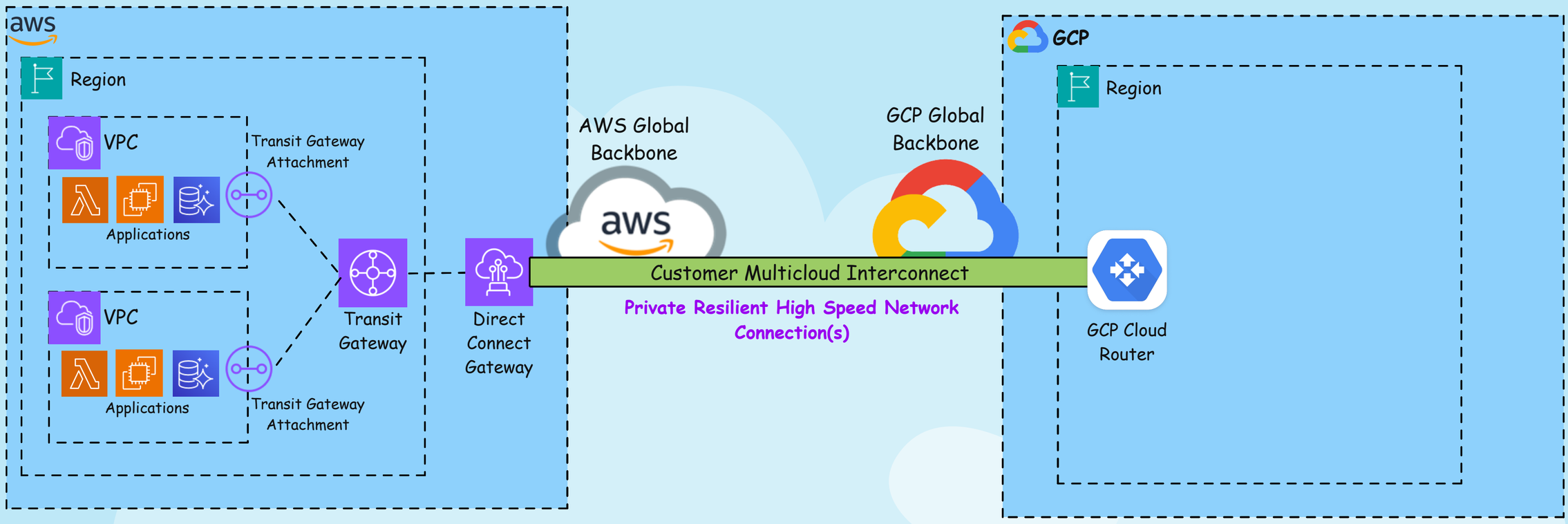


- Run Lambda on EC2 using Managed Instances!
- You can see these EC2s in your AWS Account
- Select underlying hardware including GPU
- One Lambda Managed Instance runs multiple transactions at one time (You can control how many)
- No Cold Start
- High Availability means additional EC2 leading to extra cost
- Pricing model change - No longer pay as you go, now pay for uptime of EC2 + 15% management charge + Request Charge

AWS Multicloud Interconnect

CLOUD WITH RAJ
www.cloudwithraj.com

Re:Invent 2025 Announcement



AWS Interconnect enables you to create high-speed network connections between your AWS Virtual Private Clouds (VPCs) and your VPCs on other public clouds. The service is also designed to provision capacity in minutes, and traffic is encrypted.

In this example, on AWS side, Transit Gateway Attachments are attached to your VPCs. These Transit Gateway Attachments are associated with a Transit Gateway (Don't make this mistake of saying in interview that Transit Gateway is attached to VPC, it's NOT).

This Transit Gateway sends traffic to Direct Connect gateway. From here the new enhancements takes effect!

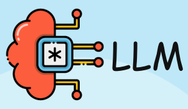
AWS and GCP (Azure coming later) has created dedicated high speed resilient network connections between AWS datacenters and Google datacenters. No traffic goes via public internet.

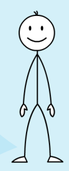
Traffic goes through this Customer Multicloud Interconnect Private connection, and can land in GCP network construct such as GCP Cloud Router.

Bedrock Agentcore Gateway

CLOUD WITH RAJ
www.cloudwithraj.com

BEFORE





User query that requires multiple MCP Tools

Incorrect/incomplete answer

Agent with MCP Capabilities

MCP Protocol

Tool(s) Coded with MCP Standards

MCP Protocol

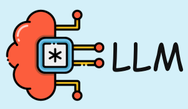
Existing Business APIs/Codes without MCP

API in API Gateway

Lambda

Existing APIs/Code need to be updated with MCP Standard (HUGE EFFORT)

AFTER RE:INVENT 2025



User query that requires multiple MCP Tools

Correct Answer

Agent with MCP Capabilities

Agent calls Agentcore Gateway for MCP Tools

MCP Protocol

Agentcore Gateway

Wrap existing APIs/Lambdas/Endpoints with MCP standard, no code change needed! MCPify ALL existing APIs/Codes

Tool(s) Coded with MCP Standards

Existing Business APIs/Codes without MCP

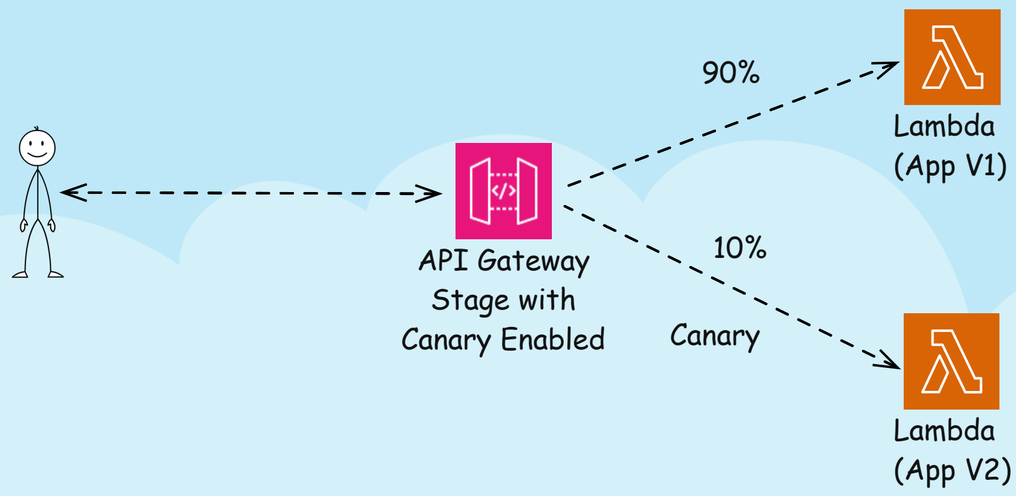
API in API Gateway

Lambda

Canary Deployment vs Blue/Green Vs Rolling Deployment

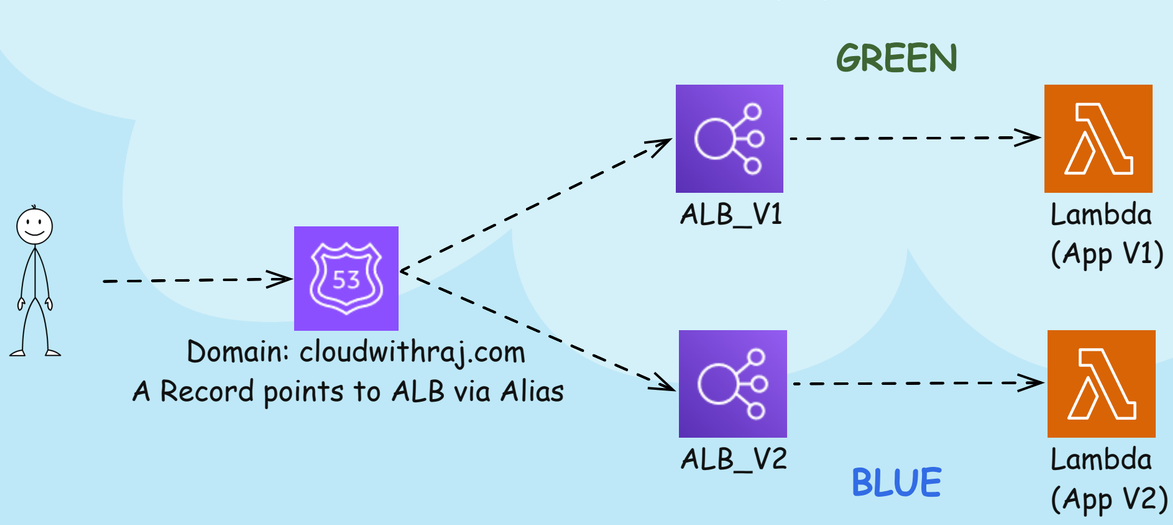
CLOUD WITH RAJ
www.cloudwithraj.com

Canary Deployment



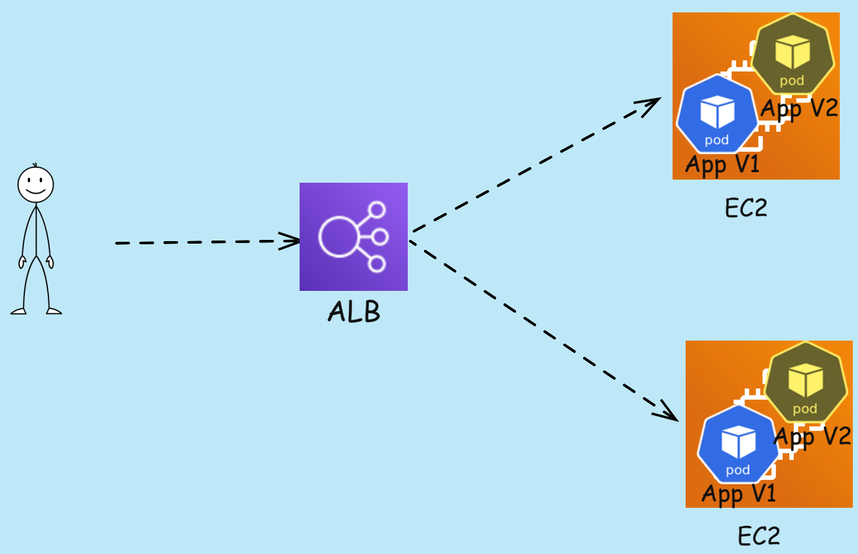
Traffic split between V1 and V2. Can be transitioned gradually to V2

Blue/Green Deployment



Traffic switched entirely from Green to Blue. Both running at same time.
Can be rolled back and forth

Rolling Deployment



Replace old versions with new versions in small batches